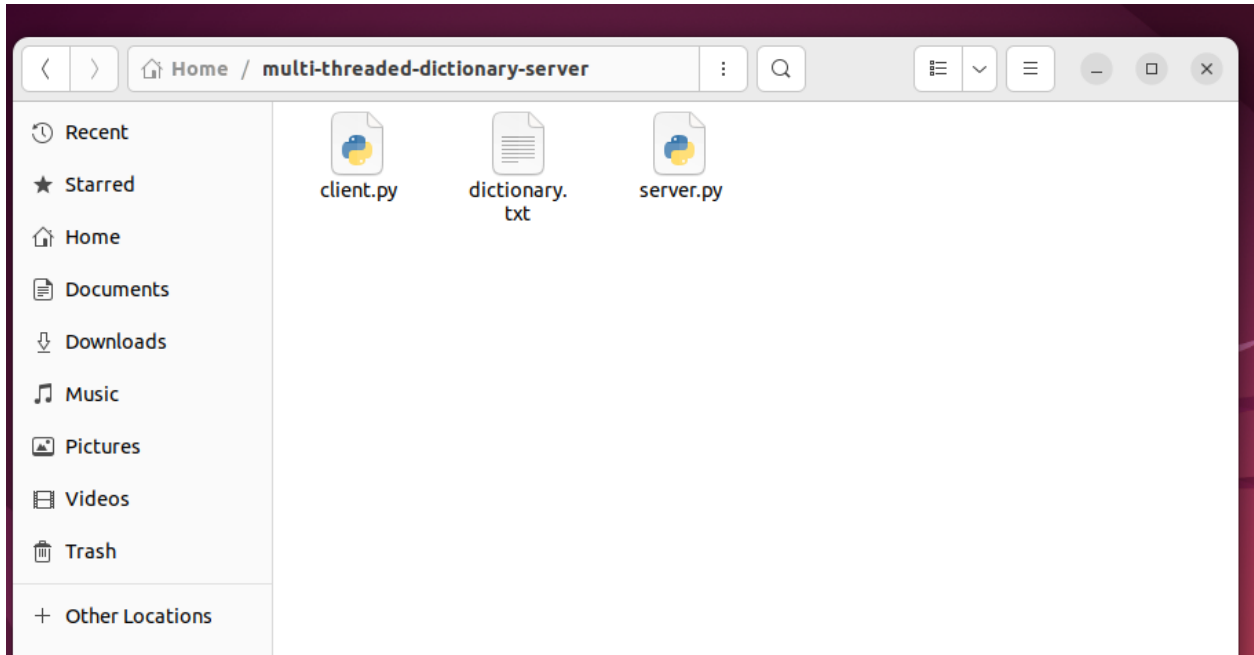


Multi-threaded Dictionary Server

In the beginning I used **Ubuntu** to achieve this assignment.

So first I created a new directory called **multi-threaded-dictionary-server** and put all new files that I should use in it (client.py, dictionary.txt, server.py).



Start with **Client.py**: creates a simple client application that allows users to search for the meaning of words, it connects to a server on a specified port, sends the search term as JSON data, and displays the received meaning or error message in the GUI.

dictionary.txt: contains all of words definition.

server.py: provides a basic server application that can be used by client applications to query a dictionary stored on the server machine.

Multi-threaded Dictionary Server

Now let's break down each code:

Client.py

```
client.py  x  server.py  x  dictionary.txt  x
1 import socket
2 import json
3 import tkinter as tk
4 HOST = '0.0.0.0'
5 PORT=5000
6
7 def search_word(word):
8     with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
9         s.connect((HOST,PORT))
10        request = {'word': word}
11        request_json = json.dumps(request)
12        s.sendall(request_json.encode())
13
14        data=s.recv(1024).decode()
15        response = json.loads(data)
16        if 'meaning' in response:
17            meaning_label.config(text=response['meaning'])
18        else:
19            error_label.config(text=response['error'])
20 window = tk.Tk()
21 window.title("Dictionary Client")
22 search_entry = tk.Entry(window)
23 search_entry.pack(padx=10,pady=10)
24
25 search_button = tk.Button(window, text="search", command=lambda: search_word(search_entry.get()))
26 search_button.pack(padx=10,pady=10)
27
28 meaning_label =tk.Label(window, text="")
29 meaning_label.pack()
30
31 error_label = tk.Label(window, text="")
32 error_label.pack()
33
34 window.mainloop()
```

import

```
import socket
import json
import tkinter as tk
```

- **socket**: This module allows creating sockets for network communication.
- **json**: This module helps working with JSON data format (JavaScript Object Notation).
- **tkinter as tk**: This module provides functions for creating graphical user interfaces (GUI) in Python. It's imported as **tk** for shorter use within the code.

Multi-threaded Dictionary Server

Host & Port

```
HOST = '0.0.0.0'  
PORT = 5000
```

- **HOST**: This variable stores the IP address of the server the client will connect to. Here, it's set to **'0.0.0.0'**, which means it will listen on all available interfaces.
 - **PORT**: This variable defines the port number on which the server listens for incoming connections. Here, it's set to **5000**.
-

GUI

```
window = tk.Tk()  
window.title("Dictionary Client")  
search_entry = tk.Entry(window)  
search_entry.pack(padx=10, pady=10)  
search_button = tk.Button(window, text="search", command=lambda: search_word(search_entry.get()))  
search_button.pack(padx=10, pady=10)  
meaning_label = tk.Label(window, text="")  
meaning_label.pack()  
error_label = tk.Label(window, text="")  
error_label.pack()  
window.mainloop()
```

- It creates a main application window using **tk.Tk()**.
- It sets the title of the window to "Dictionary Client" using **window.title("Dictionary Client")**.
- It creates an entry widget (**search_entry**) using **tk.Entry(window)** This allows the user to type in a word to search.
- It arranges the **search_entry** widget within the window using **search_entry.pack(padx=10, pady=10)** **padx** and **pady** add padding around the widget.
- It creates a button widget (**search_button**) using **tk.Button(window, text="search", command=lambda: search_word(search_entry.get()))**
 - The button text is set to "search".
 - Clicking the button triggers the **search_word** function with the text entered in the **search_entry** as an argument (**search_entry.get()**) This is achieved using a lambda function.
- It arranges the **search_button** widget within the window using **search_button.pack(padx=10, pady=10)**.
- It creates two label widgets (**meaning_label** and **error_label**) using **tk.Label(window, text="")** Initially, their text is set to an empty string.
- It arranges the **meaning_label** widget within the window using **meaning_label.pack()** .
- It arranges the **error_label** widget within the window using **error_label.pack()**.

Multi-threaded Dictionary Server

- Finally, it enters the main loop of the `Tkinter` event listener using `window.mainloop()`. This keeps the window open and responds to user interactions like button clicks or typing in the entry widget.

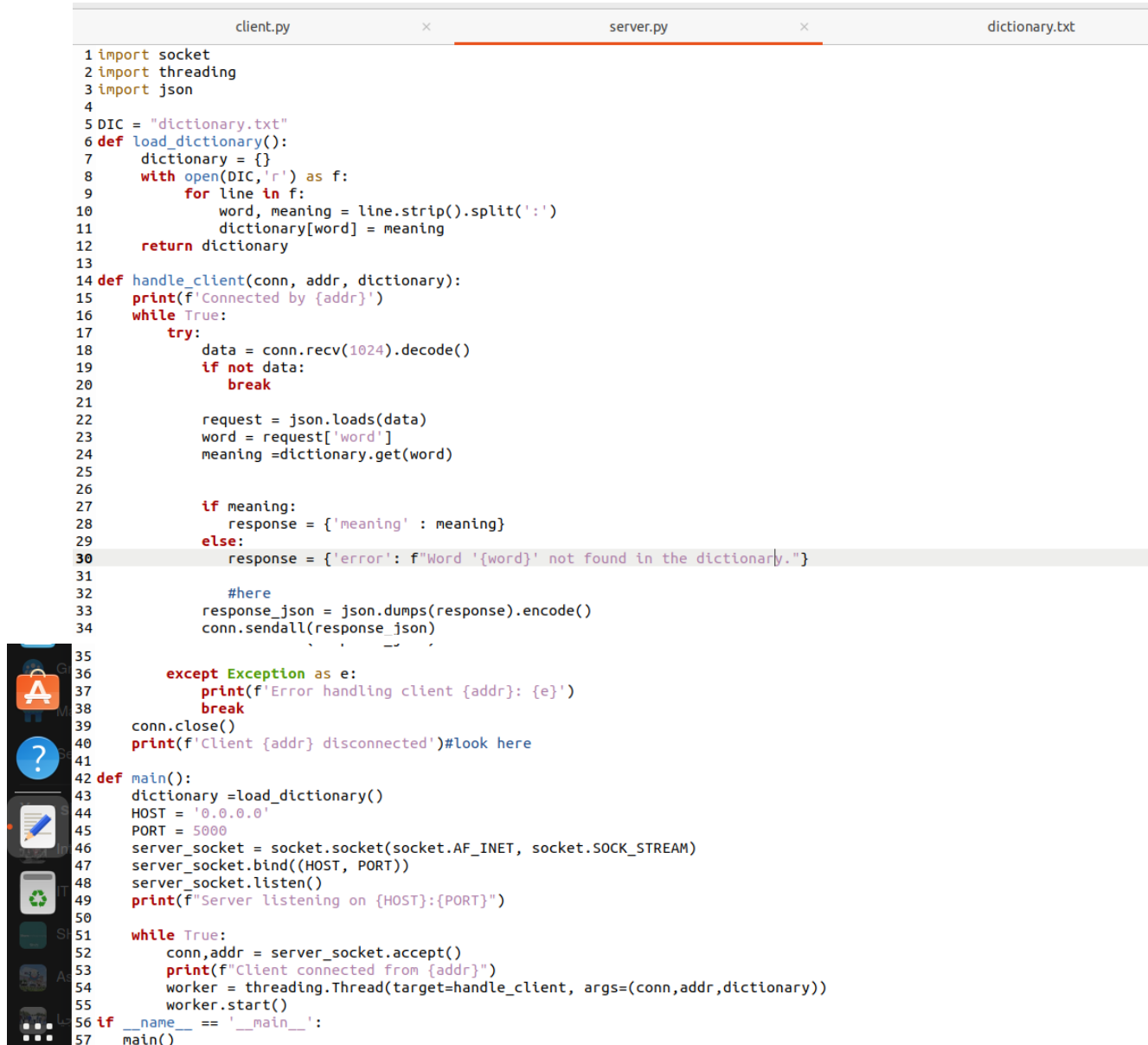
Search_word(word)

```
def search_word(word):  
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:  
        s.connect((HOST, PORT))  
        request = {'word': word}  
        request_json = json.dumps(request)  
        s.sendall(request_json.encode())  
        data = s.recv(1024).decode()  
        response = json.loads(data)  
        if 'meaning' in response:  
            meaning_label.config(text=response['meaning'])  
        else:  
            error_label.config(text=response['error'])
```

- This function takes a word argument (`word`).
- It creates a socket object using `socket.socket(socket.AF_INET, socket.SOCK_STREAM)`. This creates a TCP socket (reliable, stream-based connection).
- It connects the socket to the server using `s.connect((HOST, PORT))`.
- It creates a dictionary object `request` with a key `'word'` and sets its value to the argument `word`.
- It converts the `request` dictionary into a **JSON** string using `json.dumps(request)`.
- It sends the JSON-encoded request to the server using `s.sendall(request_json.encode())`.
- It receives data from the server up to 1024 bytes using `s.recv(1024)`.
- It decodes the received data using `data.decode()`.
- It converts the JSON-decoded data back into a Python dictionary using `json.loads(data)`.
- It checks if the response dictionary has a key `'meaning'`.
 - If yes, it updates the text of the `meaning_label` with the value of the `'meaning'` key using `meaning_label.config(text=response['meaning'])`.
 - If not, it updates the text of the `error_label` with the value of the `'error'` key using `error_label.config(text=response['error'])`.

Server.py

Multi-threaded Dictionary Server



```
1 import socket
2 import threading
3 import json
4
5 DIC = "dictionary.txt"
6 def load_dictionary():
7     dictionary = {}
8     with open(DIC, 'r') as f:
9         for line in f:
10             word, meaning = line.strip().split(':')
11             dictionary[word] = meaning
12     return dictionary
13
14 def handle_client(conn, addr, dictionary):
15     print(f'Connected by {addr}')
16     while True:
17         try:
18             data = conn.recv(1024).decode()
19             if not data:
20                 break
21
22             request = json.loads(data)
23             word = request['word']
24             meaning = dictionary.get(word)
25
26
27             if meaning:
28                 response = {'meaning': meaning}
29             else:
30                 response = {'error': f'Word '{word}' not found in the dictionary.'}
31
32             #here
33             response_json = json.dumps(response).encode()
34             conn.sendall(response_json)
35
36         except Exception as e:
37             print(f'Error handling client {addr}: {e}')
38             break
39     conn.close()
40     print(f'Client {addr} disconnected')#look here
41
42 def main():
43     dictionary = load_dictionary()
44     HOST = '0.0.0.0'
45     PORT = 5000
46     server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
47     server_socket.bind((HOST, PORT))
48     server_socket.listen()
49     print(f"Server listening on {HOST}:{PORT}")
50
51     while True:
52         conn, addr = server_socket.accept()
53         print(f"Client connected from {addr}")
54         worker = threading.Thread(target=handle_client, args=(conn, addr, dictionary))
55         worker.start()
56
57 if __name__ == '__main__':
58     main()
```

Import

```
import socket
import threading
import json
```

- **socket**: This module allows creating sockets for network communication.
- **threading**: This module provides functionality for working with threads.
- **json**: This module helps working with JSON data format (JavaScript Object Notation).

Multi-threaded Dictionary Server

dictionary

DIC="dictionary.txt"

- **DIC**: This variable stores the path to the dictionary file ("dictionary.txt").
-

load_dictionary()

```
def load_dictionary():
    dictionary = {}
    with open(DIC, 'r') as f:
        for line in f:
            word, meaning = line.strip().split(':')
            dictionary[word] = meaning
    return dictionary
```

- This function is responsible for loading the dictionary from a text file.
 - It creates an empty dictionary `dictionary = {}`.
 - It opens the file specified by **DIC** in read mode ('r').
 - It iterates over each line in the file.
 - It splits the line by the colon (:) delimiter, assuming each line is formatted as "**word: meaning**".
 - It extracts the word and its meaning by assigning them to separate variables (**word and meaning**).
 - It adds the word as a key and the meaning as the value to the dictionary.
 - It returns to the populated dictionary.
-

handle_client(conn, addr, dictionary)

```
def handle_client(conn, addr, dictionary):
    print(f'Connected by {addr}')
```

- This function takes three arguments:
 - **conn**: A socket object representing the connection to the client.
 - **addr**: A tuple containing the client's IP address and port number.
 - **dictionary**: The dictionary loaded from the file.
- It prints a message indicating a client connection `print(f'Connected by {addr}')`.

```
while True:
    try:
        data = conn.recv(1024).decode()
        if not data:
            break
        request = json.loads(data)
        word = request['word']
```

Multi-threaded Dictionary Server

```
meaning=dictionary.get(word)
if meaning:
    response={'meaning': meaning}
else:
    response={'error': f"Word '{word}' not found in the dictionary."}
    response_json=json.dumps(response).encode()
    conn.sendall(response_json)
except Exception as e:
    print(f'Error handling client {addr}: {e}')
    break
conn.close()
print(f'Client {addr} disconnected')
```

It enters a loop that continues until there's an error or the connection closes.

- Inside the loop, it tries to receive data from the client using `conn.recv(1024)`
- It decodes the received bytes using `decode()`.
- If no data is received (`data` is empty), it breaks out of the loop assuming the connection **is closed**.
- It converts the received data from JSON format into a Python dictionary using `json.loads(data)`.
- It retrieves the requested word from the JSON object using `request['word']`.
- It tries to look up the word in the dictionary using `dictionary.get(word)`.
The `get` method returns the value associated with the key if it exists, otherwise it returns `None`.
- If the word is found (`meaning` is not `None`):
 - It creates a response dictionary with a key `'meaning'` and sets its value to the meaning retrieved from the dictionary.
- If the word is not found:
 - It creates a response dictionary with a key `'error'`. The value is an error message indicating the word was not found.
- It converts the response dictionary back to JSON format using `json.dumps(response)`.
- It encodes the JSON data back into bytes using `encode()`.
- It sends the encoded response data back to the client using `conn.sendall(response_json)`.
- In case of any exception (`except` block), it prints an error message and exits the loop, assuming the connection is broken `print(f'Error handling client {addr}: {e}')`.
- Finally, it closes the client connection using `conn.close()` and prints a message indicating the client disconnection `print(f'Client {addr} disconnected')`.

main()

```
dictionary=load_dictionary()
HOST='0.0.0.0'
PORT=5000
```

Multi-threaded Dictionary Server

```
server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server_socket.bind((HOST, PORT))
server_socket.listen()
print(f"Server listening on {HOST}:{PORT}")
```

- It calls the `load_dictionary` function to load the dictionary from the file.
- It defines the server's IP address (`HOST`) as `'0.0.0.0'` (listening on all available interfaces) and port (`PORT`) as `5000`.
- It creates a server socket using `socket.socket(socket.AF_INET, socket.SOCK_STREAM)`. This creates a TCP socket (reliable, stream-based connection).
- It binds the socket to the specified address and port using `server_socket.bind((HOST, PORT))`.
- It sets the socket to listen for incoming connections using `server_socket.listen()`.
- It prints a message indicating the server is listening on the specified port `print(f"Server listening on {HOST}:{PORT}")`.

```
while True:
    conn, addr = server_socket.accept()
    print(f"Client connected from {addr}")
    worker = threading.Thread(target=handle_client, args=(conn, addr, dictionary))
    worker.start()
if __name__ == '__main__':
    main()
```

It enters an infinite loop to continuously accept new client connections.

- Inside the loop, it uses `server_socket.accept()` to accept a new connection. This method returns a new socket object (`conn`) representing the connection to the client and a tuple (`addr`) containing the client's IP address and port number.
- It prints a message indicating a new client connection.
- It creates a thread using `threading.Thread(target=handle_client, args=(conn, addr, dictionary))`.
 - The target of the thread is set to the `handle_client` function.
- The arguments passed to the thread are:
 - `conn`: The socket object representing the client connection.
 - `addr`: The tuple containing the client's IP address and port number.
 - `dictionary`: The dictionary loaded from the file (shared between the main thread and the worker thread).
- It starts the thread using `worker.start()`. This allows the server to handle multiple clients concurrently without blocking the main loop.

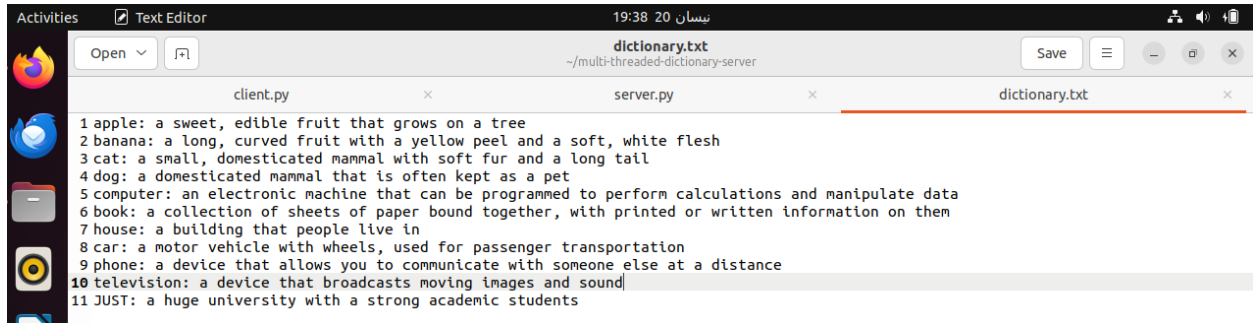
Conditional Execution (if `__name__ == '__main__':`):

- This block ensures the main function is only executed when the script is run directly and not imported as a module.

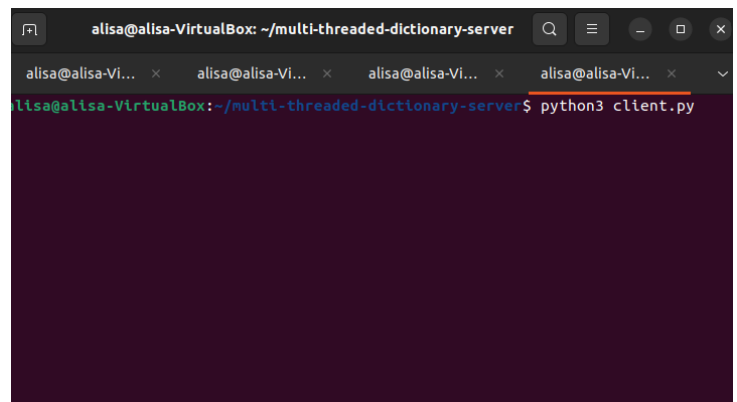
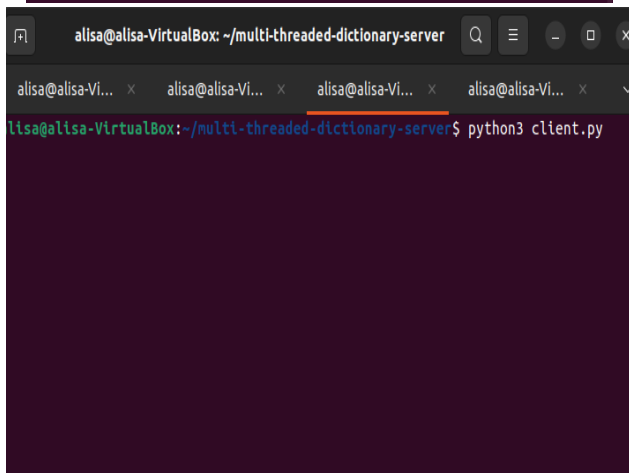
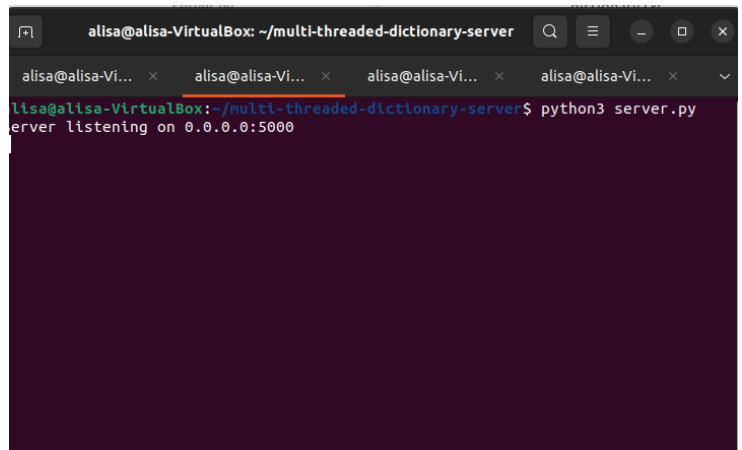
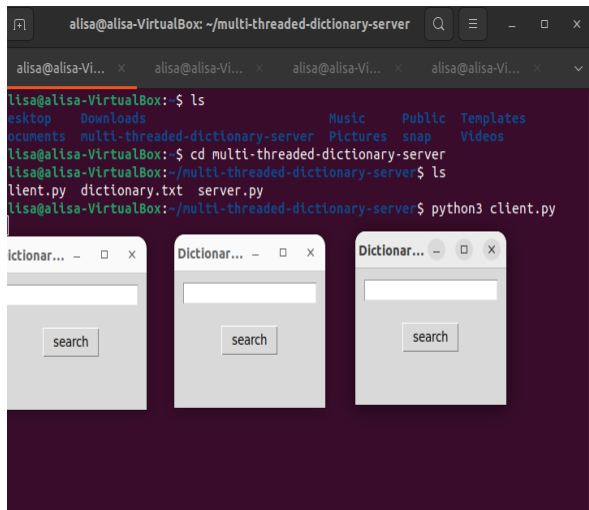
Multi-threaded Dictionary Server

- Inside this block, it calls the main function to start the server.

Dictionary:

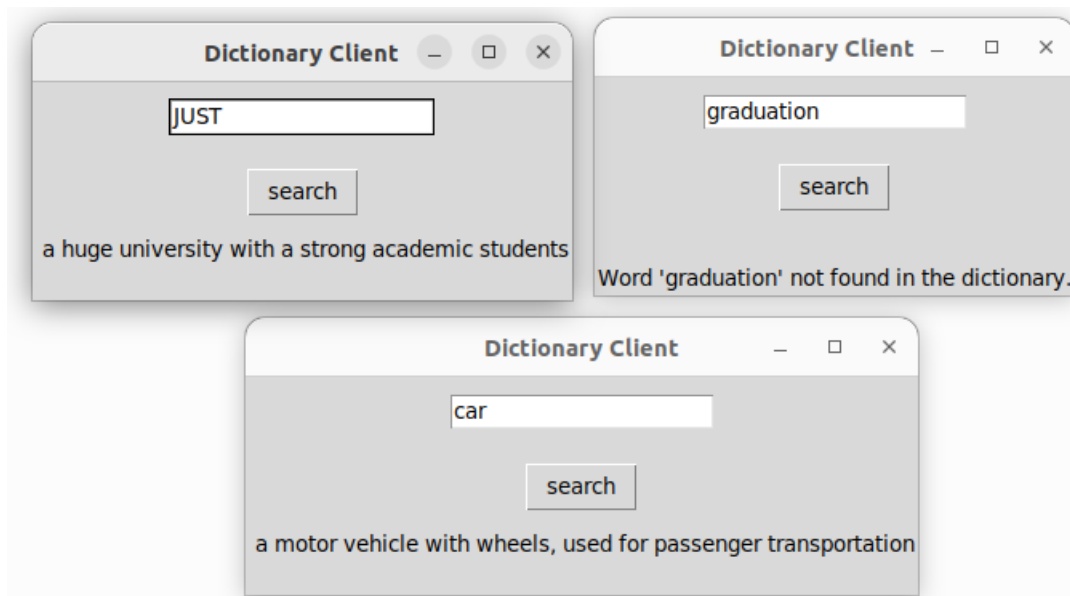


Now let start it:



Multi-threaded Dictionary Server

Here I run all client.py three times with different cmd in parallel and run server.py to connect with them so now I going to put two words exist inside our dictionary and one doesn't, so let see.



```
alisa@alisa-VirtualBox: ~/multi-threaded-dictionary-server
alisa@alisa-Vi... x  alisa@alisa-Vi... x  alisa@alisa-Vi... x  alisa@alisa-Vi... x
alisa@alisa-VirtualBox:~/multi-threaded-dictionary-server$ python3 server.py
Server listening on 0.0.0.0:5000
Client connected from ('127.0.0.1', 48336)
Connected by ('127.0.0.1', 48336)
Client ('127.0.0.1', 48336) disconnected
Client connected from ('127.0.0.1', 51372)
Connected by ('127.0.0.1', 51372)
Client ('127.0.0.1', 51372) disconnected
Client connected from ('127.0.0.1', 53860)
Connected by ('127.0.0.1', 53860)
Client ('127.0.0.1', 53860) disconnected
```